VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

ADVANCED PROGRAMMING - CO2039

ASSIGNMENT 2

INTERPRETING INTERSTELLAR PROGRAM

Author: MEng. Nguyen Minh Tam

HO CHI MINH CITY, 04/2026

ASSIGNMENT'S SPECIFICATION

Version 1.0

1 Assignment's outcome

Upon completion of this assignment, students will be able to:

- Parse and interpret structured token-based inputs according to a formally defined specification.
- Implement evaluation mechanisms for a functional language, including handling of expressions, operators, and variable bindings.
- Apply concepts of functional programming such as lambda abstractions, substitution, and non-strict (call-by-name) evaluation.
- Design and implement recursive evaluation strategies while respecting computational constraints.

2 Introduction

Programming languages can vary significantly in how they represent and evaluate expressions. Some languages, especially functional ones, rely on compact symbolic representations and formal evaluation rules that may not be immediately intuitive. The Interstellar Functional Program is one such language, where a program is expressed as sequences of tokens and evaluated using a call-by-name strategy.

In this assignment, students will design and implement an interpreter for this language. The interpreter must parse tokens, identify their types and evaluate expressions according to the specified semantics. This includes handling unary and binary operations, conditional expressions, and function application through substitution. The purpose of this assignment is to provide hands-on experience with core concepts in functional programming and language interpretation.

3 Interstellar Functional Program (IFP)

An *Interstellar Functional Program* (IFP) is a sequence of **space-separated tokens**. Each token is a non-empty string composed only of printable ASCII characters from code 33 (!) to 126 (~). Therefore, each character belongs to a set of 94 possible symbols. Each token has two parts:

- **Indicator:** the first character of the token

- **Body:** the remaining characters (possibly empty)

The indicator determines how the token is interpreted. The following subsections define all supported token types.

3.1 Boolean Constants

- `T` represents the boolean value `true`
- `F` represents the boolean value `false`

The body must be empty.

3.2 Integer Values

Tokens beginning with `I` represent integers.

- The body must be non-empty
- The body is interpreted as a base-94 number

Each character corresponds to a digit:

- `!` represents 0
- `"` represents 1
- ...
- `~` represents 93

Example: `I/6` represents the number `1337`.

3.3 String Values

Tokens beginning with `S` represent strings. Each character in the body is decoded using the following ordered alphabet:

```
abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789
!"#$%&'()*+,-./:;<=>?@[\]^_`|~<space><newline>
```

- `<space>` corresponds to the space character

- `<newline>` corresponds to a newline character

Example: `SB%,,/}Q/2,$_` means `Hello World!`

3.4 Unary Operators

Tokens beginning with `U` represent unary operators.

- The body must contain exactly one character
- The token is followed by one valid IFP expression

Op	Description	Example
`-`	Integer negation	`U- I$` → `-3`
`!`	Boolean negation	`U! T` → `false`
`#`	Convert string to integer	`U# S4%34` → `15818151`
`$`	Convert integer to string	`U$ I4%34` → `test`

3.5 Binary Operators

Tokens beginning with `B` represent binary operators.

- The body must contain exactly one character
- The token is followed by two valid IFP expressions

Op	Description	Example
+	Addition	B+ I# I$ → 5
-	Subtraction	B- I$ I# → 1
*	Multiplication	B* I$ I# → 6
/	Division (truncate toward 0)	B/ U- I(I# → -3
%	Modulo	B% U- I(I# → -1
<	Less than	B< I$ I# → false
>	Greater than	B> I$ I# → true
=	Equality	B= I$ I# → false
|	Boolean OR	B| T F → true
&	Boolean AND	B& T F → false
.	String concatenation	B. S4% S34 → test
T	Take first x characters	BT I$ S4%34 → tes
D	Drop first x characters	BD I$ S4%34 → t
$	Function application	See Section 3.8

3.6 Conditional Expression

The token ? represents a conditional expression.

- It is followed by three expressions: condition, then-branch, else-branch
- The condition must evaluate to a boolean value

Example: ? B> I# I$ S9%3 S./ evaluates to no.

3.7 Lambda Abstractions and Variables

- L defines a lambda abstraction
- v represents a variable

The body encodes a variable identifier using base-94. A lambda expression binds a variable and has one body expression.

Example:

```
L# <expression>
v#
```

3.8 Function Application

The operator `B$` applies a function to an argument.

- If the left-hand side evaluates to a lambda abstraction, the argument is substituted into the body
- Substitution must avoid variable capture

Example: `B$ B$ L# L$ v# B. SB%,,/ S}Q/2,$_ IK` represents the program (e.g. in Haskell style):

```
((\v2 -> \v3 -> v2) ("Hello" . " World!")) 42
```

which would evaluate to the string `Hello World!`

3.9 Evaluation Strategy

Evaluation follows a **call-by-name** strategy:

- Arguments are not evaluated before substitution
- Expressions are evaluated only when needed
- If a variable appears multiple times, its expression may be evaluated multiple times

Example reduction:

```
B$ L# B$ L" B+ v" v" B* I$ I# v8
=> B$ L" B+ v" v" B* I$ I#
=> B+ B* I$ I# B* I$ I#
=> B+ I' I'
=> I-
```

3.10 Execution Limits

- Evaluation is limited to at most 10,000,000 beta-reduction steps
- Built-in operators are evaluated strictly (except function application)
- Built-in operations do not count toward the reduction limit

Example:

```
B$ B$ L" B$ L# B$ v" B$ v# v# L# B$ v" B$ v# v# L" L# ? B= v# I! I"
B$ L$ B+ B$ v" v$ B$ v" v$ B- v# I" I%
```

This expression evaluates to 16 using 109 reduction steps.

3.11 Remarks

- All input programs are guaranteed to be syntactically valid
- Only the constructs defined above are required
- Additional constructs may exist but are not part of this assignment

4 Requirements

- The conditional operator (`?`) must evaluate only the selected branch. The non-selected branch must not be evaluated under any circumstances.
- The unary operators `U#` and `U$` must correctly convert between strings and integers using the same base-94 encoding scheme.
- The program must correctly handle repeated evaluation of expressions. If an expression is substituted multiple times, it must be evaluated each time independently.
- The implementation must enforce the maximum number of allowed beta reductions. Evaluation must terminate if the number of reductions exceeds the given limit.
- You are not allowed to use built-in expression evaluators, scripting engines, or reflection mechanisms. The evaluation logic must be implemented manually.
- Be careful when implementing recursion or deep evaluation. Improper handling may lead to stack overflow or excessive runtime.

5 Submission

Students must submit their code via the LMS. Students are allowed to have a **maximum of up to 5 submissions**; however, only the final submission will be used for grading and evaluation.

Due to the possibility of system overload when many students submit simultaneously, it is strongly recommended to submit as early as possible. Late submissions are the student's responsibility. Once the submission deadline has passed, the system will be closed and no further submissions will be accepted. Submissions through other means will not be considered under any circumstances.

6 Other Regulations

- Students must complete this assignment independently and must not allow others to copy their work. Any violation will be treated as academic dishonesty in accordance with university regulations.
- All decisions made by the instructors responsible for this assignment are final.
- Test cases will not be disclosed after grading.

7 Changelog

——————————END——————————